

SECTION 16 — SELENIUM ADVANCED SCENARIOS (Senior SDET Masterclass)

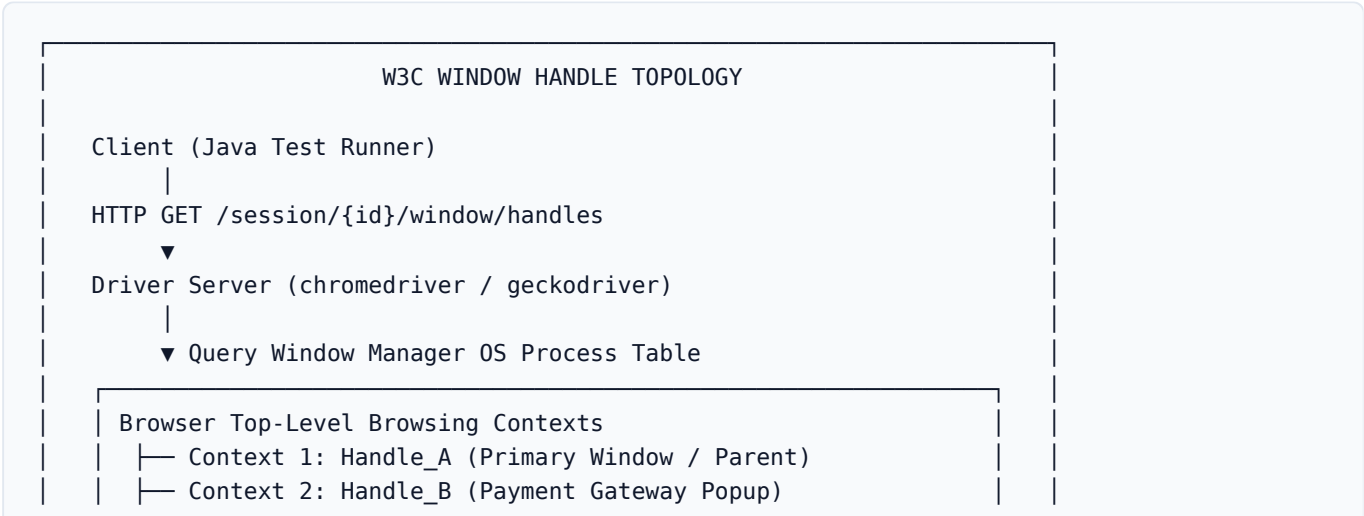
Topics Covered

- 16.1 Multi-Window & Tab Handling (Window Handles, Set Traversal, Asynchronous Popups)
- 16.2 Nested Frames & iframes (Context Trees, Switching Strategies, `parentFrame()` vs. `defaultContent()`)
- 16.3 JavaScript Alerts, Confirms, Prompts & Native HTTP Basic Authentication
- 16.4 Complex Mouse & Keyboard Actions (W3C Actions API, Hover, Drag & Drop, Composite Sequences)
- 16.5 JavaScript Executor Hacks (DOM Bypasses, Smooth Scrolling, Event Dispatching, Style Injections)
- 16.6 Cookie & Session Management (Fast-Login Bypasses, State Injection, Local & Session Storage)
- 16.7 SSL Certificate Errors & Insecure Content Options (W3C Capabilities Across Browsers)
- 16.8 Geolocation, Microphone, Camera & Push Notification Permissions (Options & CDP Automation)
- 16.9 Headless Execution Quirks (`--headless=new` , GPU Fallbacks, Viewport Collapse, Anti-Bot Headers)
- 16.10 Network Throttling & Offline Simulation via Selenium 4 CDP (Chrome DevTools Protocol)
- 16.11 Mobile Device Emulation (Device Metrics, Touch Emulation, User-Agent Spoofing)
- 16.12 Screen Recording & Automated Failure Screenshots (TestNG Listeners & CI Evidence Pipelines)
- 16.13 High-Stakes Senior Advanced Scenarios Interview Questions & Spoken Solutions

16.1 Multi-Window & Tab Handling

1. Low-Level Mechanics & Protocol Architecture

Under the W3C WebDriver specification, browsers do not differentiate fundamentally between a browser "tab" and an operating system "window"; both are represented as a **top-level browsing context**. Each top-level context is assigned a globally unique, alphanumeric identifier string known as a **Window Handle** (e.g., `CDwindow-3C8159E444B2C300F951475A7712E8B9`).



```

└─ Context 3: Handle_C (OAuth Provider Tab)
    |
    ▼ Returns JSON Array: ["Handle_A", "Handle_B", "Handle_C"]
    WebDriver Engine: Converts to java.util.Set<String> (UNORDERED!)

```

[!WARNING]

The Non-Deterministic Ordering Trap (`Set<String>`):

`driver.getWindowHandles()` returns a `java.util.Set<String>`. In the Java implementation, this Set **DOES NOT guarantee insertion order or visual tab order**. You cannot assume that index `0` is the parent and index `1` is the new popup. Relying on positional iteration (`iterator.next()`) causes intermittent CI test failures when the JVM or driver engine rearranges handle sets.

2. Enterprise Failure Scenarios & Window Synchronization

- Window Creation Lag:** A button triggers `window.open()` via JavaScript. The test runner immediately invokes `driver.getWindowHandles()`. Because the OS thread and browser engine take 50–300ms to spawn the process and initialize the WebContents, `getWindowHandles().size()` remains `1`, causing `NoSuchWindowException` or index out of bounds.
- Handle Context Leaks:** A child tab finishes its task (e.g., completing 3D-Secure payment). The test calls `driver.close()`. The child window terminates, but the driver's active context pointer still points to the destroyed window! Subsequent commands throw `NoSuchWindowException: no such window: target window already closed`.

3. Production-Ready Window & Tab Manager

```

package com.enterprise.automation.core.window;

import org.openqa.selenium.*;
import org.openqa.selenium.support.ui.FluentWait;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import java.time.Duration;
import java.util.Set;
import java.util.function.Function;

public final class WindowManager {

    private static final Logger log = LoggerFactory.getLogger(WindowManager.class);
    private final WebDriver driver;
    private final Duration defaultTimeout;

    public WindowManager(WebDriver driver) {
        this(driver, Duration.ofSeconds(10));
    }

    public WindowManager(WebDriver driver, Duration defaultTimeout) {

```

```

        this.driver = driver;
        this.defaultTimeout = defaultTimeout;
    }

    /**
     * Executes an action that triggers a new window/tab, waits for the handle count
     * to increment, switches to the new window, executes the action, closes it, and restores pa
     */
    public <T> T executeInNewWindow(Runnable triggerAction, Function<WebDriver, T> childWindowTa
        String parentHandle = driver.getWindowHandle();
        Set<String> initialHandles = driver.getWindowHandles();

        log.info("Triggering new window creation. Base handle count: {}", initialHandles.size())
        triggerAction.run();

        // Explicitly wait until the total window count increments
        String newWindowHandle = waitForNewWindowHandle(initialHandles);

        try {
            driver.switchTo().window(newWindowHandle);
            log.info("Switched to child window. Title: [{}], URL: [{}]", driver.getTitle(), driv
            return childWindowTask.apply(driver);
        } finally {
            // Defensive teardown: Close child window and guarantee return to parent
            try {
                if (driver.getWindowHandles().contains(newWindowHandle)) {
                    log.info("Closing child window: {}", newWindowHandle);
                    driver.close();
                }
            } catch (Exception e) {
                log.warn("Child window already closed or unreachable: {}", e.getMessage());
            }
            log.info("Reverting driver focus to parent window handle: {}", parentHandle);
            driver.switchTo().window(parentHandle);
        }
    }

    /**
     * Switches to a window matching a specific title substring, with polling synchronization.
     */
    public boolean switchToWindowByTitle(String titleSubstring) {
        return new FluentWait<>(driver)
            .withTimeout(defaultTimeout)
            .pollingEvery(Duration.ofMillis(250))
            .ignoring(WebDriverException.class)
            .until(d -> {
                for (String handle : d.getWindowHandles()) {
                    d.switchTo().window(handle);
                    if (d.getTitle() != null && d.getTitle().contains(titleSubstring)) {
                        log.info("Successfully matched and switched to window with title: {}
                        return true;
                    }
                }
            })
            .return false;
    });
}

```

```

/**
 * Switches to a window matching a URL regular expression.
 */
public boolean switchToWindowByUrlPattern(String urlPattern) {
    return new FluentWait<>(driver)
        .withTimeout(defaultTimeout)
        .pollingEvery(Duration.ofMillis(250))
        .until(d -> {
            for (String handle : d.getWindowHandles()) {
                d.switchTo().window(handle);
                if (d.getCurrentUrl() != null && d.getCurrentUrl().matches(urlPattern))
                    log.info("Matched window handle {} by URL pattern: {}", handle, urlP
                    return true;
            }
        })
        .return false;
    });
}

private String waitForNewWindowHandle(Set<String> existingHandles) {
    return new FluentWait<>(driver)
        .withTimeout(defaultTimeout)
        .pollingEvery(Duration.ofMillis(200))
        .withMessage("Timed out waiting for new window handle to spawn")
        .until(d -> {
            Set<String> currentHandles = d.getWindowHandles();
            currentHandles.removeAll(existingHandles);
            return currentHandles.isEmpty() ? null : currentHandles.iterator().next();
        });
}
}

```

16.2 Nested Frames & iframes

1. Frame Hierarchy & DOM Boundary Isolation

An `iframe` or `frame` introduces an entirely separate HTML document nested inside a parent document. Each frame possesses its own:

- Global `window` and `document` object instances.
- Independent CSS rendering context.
- Discrete JavaScript runtime sandbox (governed by the Same-Origin Policy when hosted on differing domains).

Top-Level Document (defaultContent)



```
|
└─ <button id="submit-order">
```

Must navigate step-by-step

2. Switching Rules Under W3C WebDriver

- `driver.switchTo().frame(int index)` : **Flaky anti-pattern**. Frame indices depend on visual rendering order and dynamically injected tracking iframes (Google Tag Manager, Segment, Hotjar), which shift index numbers nondeterministically.
- `driver.switchTo().frame(String nameOrId)` : Fast, but fails if the iframe only has dynamic or generated IDs.
- `driver.switchTo().frame(WebElement element)` : **Gold Standard**. Uses an explicitly located and validated element reference.
- `driver.switchTo().parentFrame()` : Steps up exactly one level in the frame hierarchy (e.g., from `stripe-card-element` to `payment-host`).
- `driver.switchTo().defaultContent()` : Immediately jumps from any arbitrary nesting depth back to the topmost parent document.

3. Production-Ready Recursive Frame Traversal Engine

```
package com.enterprise.automation.core.frames;

import org.openqa.selenium.*;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import java.time.Duration;
import java.util.List;

public final class FrameManager {

    private static final Logger log = LoggerFactory.getLogger(FrameManager.class);
    private final WebDriver driver;
    private final WebDriverWait wait;

    public FrameManager(WebDriver driver, Duration timeout) {
        this.driver = driver;
        this.wait = new WebDriverWait(driver, timeout);
    }

    /**
     * Safely switches to a nested iframe chain by locators in sequential order.
     * Guaranteed recovery back to defaultContent on failure.
     */
    public void switchToNestedFrames(By... frameLocators) {
        driver.switchTo().defaultContent();
        for (int i = 0; i < frameLocators.length; i++) {
            By locator = frameLocators[i];
            log.info("Switching to nested frame level [{}]: {}", i, locator);
            try {
                wait.until(ExpectedConditions.frameToBeAvailableAndSwitchToIt(locator));
            } catch (TimeoutException e) {
```

```

        driver.switchTo().defaultContent();
        throw new NoSuchFrameException("Failed at nested frame depth " + i + " with loca
    }
}

/**
 * Recursively traverses every iframe in the entire DOM tree until an element matching
 * the target locator is discovered, leaving driver focus set to that frame.
 */
public boolean switchToFrameContainingElement(By targetElementLocator) {
    driver.switchTo().defaultContent();
    return scanFramesRecursively(targetElementLocator);
}

private boolean scanFramesRecursively(By targetElementLocator) {
    // Check if the target element exists in the current browsing context
    if (!driver.findElements(targetElementLocator).isEmpty()) {
        log.info("Target element found in current frame context!");
        return true;
    }

    List<WebElement> childFrames = driver.findElements(By.tagName("iframe"));
    for (int i = 0; i < childFrames.size(); i++) {
        try {
            // Re-find frames to avoid StaleElementReferenceException
            List<WebElement> currentFrames = driver.findElements(By.tagName("iframe"));
            if (i >= currentFrames.size()) break;

            driver.switchTo().frame(currentFrames.get(i));
            log.debug("Diving into child frame index: {}", i);

            if (scanFramesRecursively(targetElementLocator)) {
                return true;
            }

            // Backtrack one level up if element is not in this branch
            driver.switchTo().parentFrame();
        } catch (StaleElementReferenceException | NoSuchFrameException ex) {
            log.warn("Frame became stale or disappeared during traversal at index: {}", i);
            driver.switchTo().defaultContent();
        }
    }
    return false;
}

/**
 * Executes an operation inside a targeted frame and guarantees return to default content.
 */
public void executeInFrame(By frameLocator, Runnable action) {
    try {
        wait.until(ExpectedConditions.frameToBeAvailableAndSwitchToIt(frameLocator));
        action.run();
    } finally {
        log.debug("Restoring context to default content");
        driver.switchTo().defaultContent();
    }
}

```

```

    }
  }
}

```

16.3 JavaScript Alerts, Confirms, Prompts & Native HTTP Basic Authentication

1. Alert Types & W3C Handling

Browser dialogs created via `window.alert()`, `window.confirm()`, and `window.prompt()` are **native OS modal windows**. They pause the single-threaded JavaScript execution loop on the browser renderer process.

- They **do not exist within the DOM tree** (cannot be inspected via Chrome DevTools or targeted via XPath/CSS).
- Attempting to interact with DOM elements while an alert is active throws `UnhandledAlertException`.
- The `Alert` interface provides four primary commands:
 - `alert.accept()` : Equivalent to clicking "OK".
 - `alert.dismiss()` : Equivalent to clicking "Cancel" or pressing `Esc`.
 - `alert.getText()` : Extracts the string content of the modal.
 - `alert.sendKeys(String keys)` : Inputs text into a `prompt()` field.

```

WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(5));
Alert alert = wait.until(ExpectedConditions.alertIsPresent());
String message = alert.getText();
alert.sendKeys("Authorized User");
alert.accept();

```

2. Global Unhandled Alert Capability

When unexpected JavaScript alerts appear asynchronously (e.g., unload handlers, session timeouts), your driver configuration must specify the W3C `unhandledPromptBehavior` capability to prevent sudden test aborts:

```

ChromeOptions options = new ChromeOptions();
// Options: ACCEPT, DISMISS, ACCEPT_AND_NOTIFY, DISMISS_AND_NOTIFY, IGNORE
options.setUnhandledPromptBehaviour(UnexpectedAlertBehaviour.DISMISS_AND_NOTIFY);

```

3. HTTP Basic & Digest Authentication Popups

HTTP 401 Unauthorized challenges trigger a native OS/browser authentication prompt. This is **not a JavaScript alert**. `driver.switchTo().alert()` will fail.

Strategy A: Selenium 4 `HasAuthentication` Interface (Recommended)

Selenium 4 leverages the Chrome DevTools Protocol / BiDi transport to intercept HTTP 401 challenges seamlessly without credential leaks in browser history:

```

import org.openqa.selenium.HasAuthentication;
import org.openqa.selenium.UsernameAndPassword;

```

```
((HasAuthentication) driver).register(UsernameAndPassword.of("admin", "SuperSecretPassword123!"))
driver.get("https://internal-service.corporate.local/metrics");
```

Strategy B: URL Embedded Credentials (Legacy Workaround)

```
// Format: https://username:password@domain.com
// Caveat: Blocked by modern Chromium versions for subresources due to RFC 3986 security deprecation
driver.get("https://admin:SuperSecretPassword123%21@internal-service.corporate.local/metrics");
```

16.4 Complex Mouse & Keyboard Actions

1. The W3C Actions API Under the Hood

In Selenium 3, the `Actions` class relied on legacy JSON Wire Protocol input commands. In Selenium 4, it complies strictly with the **W3C Actions Specification**.

- Actions are modeled as an array of **Input Sources** (Key, Pointer/Mouse, Wheel).
- Actions are dispatched in discrete **Ticks** (synchronized steps).
- Every chained call (`moveToElement` , `clickAndHold`) compiles into a low-level JSON action sequence sent via `POST /session/{id}/actions` .

W3C ACTION DISPATCH TICK

```
Tick 0: Pointer Move -> Coordinates (x: 520, y: 310) Duration: 250ms
Tick 1: Pointer Down -> Button: 0 (Left Click)
Tick 2: Pause -> Duration: 100ms
Tick 3: Pointer Move -> Coordinates (x: 820, y: 310) Duration: 500ms
Tick 4: Pointer Up -> Button: 0
```

Must invoke `.build().perform()` to compile and transmit to the driver!

2. Production-Ready Action Interactions

```
package com.enterprise.automation.core.interactions;

import org.openqa.selenium.*;
import org.openqa.selenium.interactions.Actions;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;

import java.time.Duration;

public final class AdvancedActionEngine {

    private final WebDriver driver;
    private final WebDriverWait wait;
```

```

public AdvancedActionEngine(WebDriver driver, Duration timeout) {
    this.driver = driver;
    this.wait = new WebDriverWait(driver, timeout);
}

/**
 * Smooth hover over an element with a 200ms stabilization pause.
 */
public void hoverOverElement(By locator) {
    WebElement element = wait.until(ExpectedConditions.visibilityOfElementLocated(locator));
    new Actions(driver)
        .moveToElement(element)
        .pause(Duration.ofMillis(200))
        .build()
        .perform();
}

/**
 * Context-Click (Right-Click) to trigger custom web context menus.
 */
public void rightClick(By locator) {
    WebElement target = wait.until(ExpectedConditions.elementToBeClickable(locator));
    new Actions(driver)
        .contextClick(target)
        .build()
        .perform();
}

/**
 * HTML5 Drag-and-Drop with viewport boundary safety and fallback.
 */
public void dragAndDrop(By sourceLocator, By targetLocator) {
    WebElement source = wait.until(ExpectedConditions.visibilityOfElementLocated(sourceLocator));
    WebElement target = wait.until(ExpectedConditions.visibilityOfElementLocated(targetLocator));

    // Primary: Native W3C Pointer Drag
    try {
        new Actions(driver)
            .clickAndHold(source)
            .pause(Duration.ofMillis(300))
            .moveToElement(target)
            .pause(Duration.ofMillis(300))
            .release(target)
            .build()
            .perform();
    } catch (Exception ex) {
        // Fallback for HTML5 Drag-and-Drop which ignores native mouse events in some browsers
        simulateHtml5DragAndDrop(source, target);
    }
}

/**
 * Complex Key Combination: Select All, Delete, and Type Upper-case text.
 */
public void clearAndTypeUppercase(By inputLocator, String text) {

```

```

WebElement input = wait.until(ExpectedConditions.visibilityOfElementLocated(inputLocator));
Keys cmdOrCtrl = System.getProperty("os.name").toLowerCase().contains("mac") ? Keys.COMMAND : Keys.CONTROL;

new Actions(driver)
    .click(input)
    .keyDown(cmdOrCtrl)
    .sendKeys("a")
    .keyUp(cmdOrCtrl)
    .sendKeys(Keys.BACK_SPACE)
    .keyDown(Keys.SHIFT)
    .sendKeys(text)
    .keyUp(Keys.SHIFT)
    .build()
    .perform();
}

/**
 * JavaScript synthetic drag-and-drop simulation for modern HTML5 DataTransfer APIs.
 */
private void simulateHtml5DragAndDrop(WebElement source, WebElement target) {
    String js = "function createEvent(type) {" +
        "    var event = document.createEvent('CustomEvent');" +
        "    event.initCustomEvent(type, true, true, null);" +
        "    event.dataTransfer = {" +
        "        data: {}, " +
        "        setData: function(type, val) { this.data[type] = val; }, " +
        "        getData: function(type) { return this.data[type]; } " +
        "    };" +
        "    return event;" +
        "}" +
        "var dragStart = createEvent('dragstart');" +
        "arguments[0].dispatchEvent(dragStart);" +
        "var drop = createEvent('drop');" +
        "drop.dataTransfer = dragStart.dataTransfer;" +
        "arguments[1].dispatchEvent(drop);" +
        "var dragEnd = createEvent('dragend');" +
        "dragEnd.dataTransfer = dragStart.dataTransfer;" +
        "arguments[0].dispatchEvent(dragEnd);";
    ((JavascriptExecutor) driver).executeScript(js, source, target);
}
}

```

16.5 JavaScript Executor Hacks

1. When WebDriver Native Commands Fail

WebDriver interactions (`click()` , `sendKeys()`) mirror physical user actions:

- Element must be visible (`display != none` , `visibility != hidden` , `opacity != 0`).
- Element cannot be covered by a sticky header, modal backdrop, or floating overlay.
- Element must reside within the viewport coordinates.

When non-standard frontend implementations (e.g., hidden `<input type="file">`, overlay z-index anomalies, lazy infinite scroll) block native commands, **JavaScriptExecutor** operates directly on the DOM tree, bypassing the hit-testing pipeline.

2. Enterprise JavaScript Utility Class

```
package com.enterprise.automation.core.javascript;

import org.openqa.selenium.JavascriptExecutor;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;

import java.util.Map;

public final class JavaScriptExecutorUtil {

    private final JavascriptExecutor js;

    public JavaScriptExecutorUtil(WebDriver driver) {
        this.js = (JavascriptExecutor) driver;
    }

    /**
     * Centered scrolling into view: avoids sticky headers obscuring top-aligned elements.
     */
    public void scrollIntoCenter(WebElement element) {
        js.executeScript("arguments[0].scrollIntoView({behavior: 'instant', block: 'center', inl
    }

    /**
     * Forces click on obscured elements bypassing ElementClickInterceptedException.
     */
    public void clickBypassingOverlays(WebElement element) {
        js.executeScript("arguments[0].click();", element);
    }

    /**
     * Sets value directly on hidden inputs (e.g., date pickers, hidden auth tokens).
     */
    public void setHiddenFieldValue(WebElement element, String value) {
        js.executeScript(
            "arguments[0].value = arguments[1];" +
            "arguments[0].dispatchEvent(new Event('input', { bubbles: true }));" +
            "arguments[0].dispatchEvent(new Event('change', { bubbles: true }));",
            element, value
        );
    }

    /**
     * Extracts pseudo-element CSS computed values (e.g., ::before or ::after content).
     */
    public String getPseudoElementContent(WebElement element, String pseudoSelector) {
        return (String) js.executeScript(
            "return window.getComputedStyle(arguments[0], arguments[1]).getPropertyValue('content'
            element, pseudoSelector
        );
    }
}
```

```

    );
}

/**
 * Waits for asynchronous document and pending AJAX/Fetch requests to settle.
 */
public boolean waitForPageAndNetworkReady() {
    return (Boolean) js.executeScript(
        "return document.readyState === 'complete' && " +
        "(typeof window.jQuery === 'undefined' || jQuery.active === 0);"
    );
}

/**
 * Traverses into an open Shadow DOM root using JavaScript (cross-browser compatibility).
 */
public WebElement getShadowDomElement(WebElement hostElement, String innerCssSelector) {
    return (WebElement) js.executeScript(
        "return arguments[0].shadowRoot.querySelector(arguments[1]);",
        hostElement, innerCssSelector
    );
}
}

```

16.6 Cookie & Session Management

1. The 10x Velocity Secret: API-Seeded Fast-Login

Logging in through the UI (filling email, filling password, 2FA, waiting for redirect) takes **3 to 8 seconds per test**. In a 5,000-test suite running across 20 threads, UI-based login accounts for **thousands of wasted build hours**.

Architectural Solution: Authenticate via a high-speed REST API (using RestAssured / HTTP client), obtain the bearer session tokens or JWT cookies, inject them directly into the WebDriver browsing context, and trigger a single page refresh. Time taken: **150 milliseconds**.

API-SEEDED STATE INJECTION

1. API POST /api/v1/auth/login —————> Backend Identity Provider
 ← Returns Set-Cookie: SESSIONID=9f82...
2. WebDriver: Navigate to dummy domain page (e.g., /favicon.ico)
 (Required: Browser must establish domain context before cookies)
3. WebDriver.manage().addCookie(Cookie)
 Inject: "SESSIONID", "9f82...", Domain: ".corporate.com"
4. Inject HTML5 localStorage / sessionStorage tokens via JS
5. WebDriver.navigate().to("/dashboard")
 Result: Instant authenticated session without UI login!

2. Production-Ready Session Injection Engine

```
package com.enterprise.automation.core.session;

import org.openqa.selenium.Cookie;
import org.openqa.selenium.JavascriptExecutor;
import org.openqa.selenium.WebDriver;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import java.time.Instant;
import java.util.Date;
import java.util.Map;

public final class SessionStateInjector {

    private static final Logger log = LoggerFactory.getLogger(SessionStateInjector.class);
    private final WebDriver driver;
    private final String baseDomain;

    public SessionStateInjector(WebDriver driver, String baseDomain) {
        this.driver = driver;
        this.baseDomain = baseDomain;
    }

    /**
     * Seeds the browser with full authentication state (Cookies + LocalStorage).
     */
    public void injectAuthenticationState(String sessionToken, Map<String, String> localStorageItems) {
        // Step 1: Must navigate to a page on the target origin to set cookie/storage domain scope
        String bootstrapUrl = "https://" + baseDomain + "/robots.txt";
        log.info("Bootstrapping origin context at: {}", bootstrapUrl);
        driver.get(bootstrapUrl);

        // Step 2: Inject HTTP Session Cookie
        Date expiry = Date.from(Instant.now().plusSeconds(86400));
        Cookie authCookie = new Cookie.Builder("AUTH_SESSION_ID", sessionToken)
            .domain(baseDomain)
            .path("/")
            .isSecure(true)
            .isHttpOnly(true)
            .expiresOn(expiry)
            .build();

        driver.manage().addCookie(authCookie);
        log.info("Injected secure authentication cookie: {}", authCookie.getName());

        // Step 3: Inject HTML5 Web Storage items (JWT tokens, user profile metadata)
        JavascriptExecutor js = (JavascriptExecutor) driver;
        localStorageItems.forEach((key, value) -> {
            js.executeScript("window.localStorage.setItem(arguments[0], arguments[1]);", key, value);
            log.debug("Set localStorage key: {}", key);
        });

        // Step 4: Navigate directly to the protected dashboard
        driver.navigate().to("https://" + baseDomain + "/dashboard");
    }
}
```

```

    }

    /**
     * Cleanly purges all browser state between test runs to guarantee absolute isolation.
     */
    public void purgeSessionState() {
        driver.manage().deleteAllCookies();
        JavascriptExecutor js = (JavascriptExecutor) driver;
        try {
            js.executeScript("window.localStorage.clear(); window.sessionStorage.clear();");
        } catch (Exception ignored) {
            // Context might not be an HTML document
        }
        log.info("Purged cookies, localStorage, and sessionStorage");
    }
}

```

16.7 SSL Certificate Errors & Insecure Content Options

1. The Mechanics of SSL Failures in CI

In enterprise environments, lower environments (DEV, QA, UAT) frequently run on:

- Self-signed certificates.
- Expired certificates or certificates with invalid Subject Alternative Names (SAN).
- Mixed content (HTTP assets on HTTPS pages).

By default, Chromium and Gecko drivers abort navigation with `NET::ERR_CERT_AUTHORITY_INVALID` or `SEC_ERROR_UNKNOWN_ISSUER` to protect user security.

2. Multi-Browser W3C Capabilities Configuration

Under the W3C standard, the capability `acceptInsecureCerts` is universally supported across all major browser engines:

```

package com.enterprise.automation.core.driver;

import org.openqa.selenium.chrome.ChromeOptions;
import org.openqa.selenium.edge.EdgeOptions;
import org.openqa.selenium.firefox.FirefoxOptions;

public final class SecurityOptionsFactory {

    public static ChromeOptions getInsecureChromeOptions() {
        ChromeOptions options = new ChromeOptions();
        // W3C Standard Certificate acceptance
        options.setAcceptInsecureCerts(true);
        // Chromium-specific flags to bypass enterprise certificate warnings and mixed content
        options.addArguments("--ignore-certificate-errors");
        options.addArguments("--allow-insecure-localhost");
        options.addArguments("--allow-running-insecure-content");
        return options;
    }
}

```

```

    }

    public static FirefoxOptions getInsecureFirefoxOptions() {
        FirefoxOptions options = new FirefoxOptions();
        options.setAcceptInsecureCerts(true);
        options.addPreference("security.cert_pinning.enforcement_level", 0);
        return options;
    }

    public static EdgeOptions getInsecureEdgeOptions() {
        EdgeOptions options = new EdgeOptions();
        options.setAcceptInsecureCerts(true);
        options.addArguments("--ignore-certificate-errors");
        return options;
    }
}

```

16.8 Geolocation, Microphones, and Browser Notification Permissions

1. Why Native Permission Modals Break Tests

When a web application requests HTML5 permissions (e.g., `navigator.geolocation.getCurrentPosition()` or `Notification.requestPermission()`), the browser displays an **out-of-process OS-level infobar**.

- Standard Selenium cannot interact with these prompts.
- If unhandled, the modal blocks the UI thread, causing tests to hang indefinitely until the test runner times out.

2. Headless & Static Automation Profiles (ChromeOptions)

We can pre-seed Chromium's internal profile preferences (`profile.default_content_setting_values`):

- `0` = Default / Ask
- `1` = Allow
- `2` = Block

```

import org.openqa.selenium.chrome.ChromeOptions;
import java.util.HashMap;
import java.util.Map;

ChromeOptions options = new ChromeOptions();
Map<String, Object> prefs = new HashMap<>();

// 1 = Allow, 2 = Block
prefs.put("profile.default_content_setting_values.notifications", 2); // Auto-block push notific
prefs.put("profile.default_content_setting_values.geolocation", 1); // Auto-allow geolocation
prefs.put("profile.default_content_setting_values.media_stream_mic", 1); // Auto-allow microphon
prefs.put("profile.default_content_setting_values.media_stream_camera", 1); // Auto-allow camera

options.setExperimentalOption("prefs", prefs);
options.addArguments("--use-fake-ui-for-media-stream"); // Auto-approves WebRTC audio/video feed
options.addArguments("--use-fake-device-for-media-stream"); // Generates synthetic test tone/vid

```

3. Dynamic Runtime Permission Granting via Selenium 4 CDP

```
import org.openqa.selenium.chromium.ChromiumDriver;
import java.util.List;
import java.util.Map;

// Dynamically grant permissions during active test runtime without restarting browser
((ChromiumDriver) driver).executeCdpCommand(
    "Browser.grantPermissions",
    Map.of(
        "permissions", List.of("geolocation", "notifications", "audioCapture"),
        "origin", "https://app.enterprise.internal"
    )
);

// Override precise GPS coordinates to simulate specific geographic testing locations
((ChromiumDriver) driver).executeCdpCommand(
    "Emulation.setGeolocationOverride",
    Map.of(
        "latitude", 37.7749,
        "longitude", -122.4194,
        "accuracy", 100
    )
);
```

16.9 Headless Execution Quirks

1. The Chrome 109+ Paradigm Shift: `--headless=new`

For years, running Chrome in headless mode (`--headless` or `--headless=chrome`) invoked a completely separate, stripped-down browser engine codebase that lacked extensions, printed differently, handled fonts abnormally, and rendered distinct DOM bounding boxes compared to headed Chrome.

In Chrome 109+, Google introduced `--headless=new` . This uses the **exact same rendering pipeline** as regular Chrome:

Feature	Legacy (<code>--headless</code>) vs New (<code>--new</code>)
Render Engine Architecture	Separate Divergent Minimal Codebase
Full Chromium Engine (<code>--new</code>)	Identical to Desktop Headed Chrome
Viewport Default Dimensions	800 x 600 (Causes sudden collapses)
Extension Support	Not Supported in Legacy
Full Extension Support (<code>--new</code>)	Supported
<code>`navigator.webdriver`</code> Flag	Exposed as true (Bot Detection Flag)

2. The 4 Deadly Headless Production Pitfalls

1. **The 800x600 Viewport Collapse:** Headless Chrome initializes to an 800x600 viewport. Desktop navigation menus collapse into hamburger menus! Standard locators vanish, throwing `ElementNotInteractableException`.
 - *Fix:* Explicitly set `--window-size=1920,1080`.
2. **Missing System Fonts in Linux Docker Containers:** Minimal Docker images (`alpine` , `debian-slim`) lack standard fonts (Arial, Helvetica, Roboto). Web pages render with blank text or broken layout metrics.
 - *Fix:* Install `fonts-liberation` , `fonts-noto-color-emoji` , and `ttf-mscorefonts-installer` in Dockerfiles.
3. **Shared Memory Exhaustion (`/dev/shm`):** Chrome uses `/dev/shm` for IPC. In Docker, default `/dev/shm` is 64MB, causing Chrome to crash with `SessionNotCreatedException: DevToolsActivePort file doesn't exist`.
 - *Fix:* Add `--disable-dev-shm-usage` or configure docker container with `--shm-size=2g`.
4. **Anti-Bot Fingerprint Detection:** In headless mode, Cloudflare or Akamai detects `navigator.webdriver === true` and blocks CI requests.
 - *Fix:* Exclude automation switches via `options.setExperimentalOption("excludeSwitches", List.of("enable-automation"))`.

3. Bulletproof Headless Chrome Configuration

```
package com.enterprise.automation.core.driver;

import org.openqa.selenium.chrome.ChromeOptions;
import java.util.List;

public final class HeadlessDriverConfig {

    public static ChromeOptions createProductionHeadlessOptions() {
        ChromeOptions options = new ChromeOptions();

        // Modern Headless
        options.addArguments("--headless=new");

        // Enforce full HD viewport to prevent responsive UI collapses
        options.addArguments("--window-size=1920,1080");

        // Critical CI / Docker memory and sandbox parameters
        options.addArguments("--no-sandbox");
        options.addArguments("--disable-dev-shm-usage");
        options.addArguments("--disable-gpu");

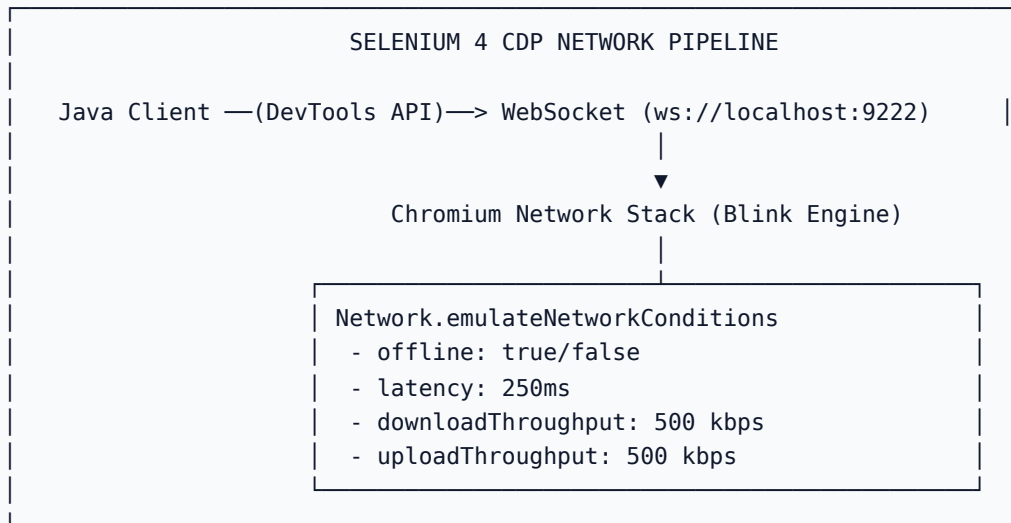
        // Prevent Bot-Detection traps on enterprise CDNs
        options.addArguments("--user-agent=Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit");
        options.setExperimentalOption("excludeSwitches", List.of("enable-automation"));
        options.setExperimentalOption("useAutomationExtension", false);

        return options;
    }
}
```

16.10 Network Throttling & Offline Simulation via Selenium 4 CDP

1. Chrome DevTools Protocol (CDP) WebSocket Architecture

Selenium 4 enables direct, bidirectional communication with the Chromium engine via WebSocket connections using the `DevTools` interface. This allows SDETs to control the browser's network layer directly, bypassing the OS.



2. Production-Ready Network Simulation Manager

```

package com.enterprise.automation.core.network;

import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.devtools.DevTools;
import org.openqa.selenium.devtools.v122.network.Network;
import org.openqa.selenium.devtools.v122.network.model.ConnectionType;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import java.util.Optional;

public final class NetworkThrottlingManager {

    private static final Logger log = LoggerFactory.getLogger(NetworkThrottlingManager.class);
    private final DevTools devTools;

    public NetworkThrottlingManager(ChromeDriver driver) {
        this.devTools = driver.getDevTools();
        this.devTools.createSession();
        this.devTools.send(Network.enable(Optional.empty(), Optional.empty(), Optional.empty()))
    }

    /**
     * Simulates complete network disconnection (Offline PWA & offline error testing).
     */
    public void simulateOffline() {
        log.warn("Enabling complete offline network simulation");
        devTools.send(Network.emulateNetworkConditions(
            true, // offline
            0, // latency (ms)
  
```

```

        0,                // downloadThroughput (bytes/s)
        0,                // uploadThroughput (bytes/s)
        Optional.of(ConnectionType.NONE),
        Optional.empty(),
        Optional.empty(),
        Optional.empty()
    ));
}

/**
 * Simulates high-latency Slow 3G network conditions.
 */
public void simulateSlow3G() {
    log.info("Throttling network to Slow 3G (Latency: 2000ms, DL: 50kbps)");
    devTools.send(Network.emulateNetworkConditions(
        false,
        2000,                // 2000 ms latency
        (50 * 1024) / 8,     // 50 kbps download
        (50 * 1024) / 8,     // 50 kbps upload
        Optional.of(ConnectionType.CELLULAR3G),
        Optional.empty(),
        Optional.empty(),
        Optional.empty()
    ));
}

/**
 * Restores unconstrained gigabit network conditions.
 */
public void resetNetwork() {
    log.info("Resetting network conditions to normal");
    devTools.send(Network.emulateNetworkConditions(
        false,
        0,
        -1,
        -1,
        Optional.of(ConnectionType.ETHERNET),
        Optional.empty(),
        Optional.empty(),
        Optional.empty()
    ));
}
}

```

16.11 Mobile Device Emulation

1. Desktop Chromium Mobile Emulation

Rather than spinning up heavy Appium mobile emulator instances for responsive web validation, Chromium's built-in `mobileEmulation` allows high-speed testing of:

- Viewport dimensions and Device Pixel Ratio (DPR).
- User-Agent strings and platform headers.

- Touch events (`ontouchstart` , `ontouchend`) instead of mouse clicks.

```
package com.enterprise.automation.core.driver;

import org.openqa.selenium.chrome.ChromeOptions;
import java.util.HashMap;
import java.util.Map;

public final class MobileEmulationFactory {

    /**
     * Emulates standard iPhone 14 Pro using standard Chrome presets.
     */
    public static ChromeOptions getIPhone14Preset() {
        ChromeOptions options = new ChromeOptions();
        Map<String, String> mobileEmulation = new HashMap<>();
        mobileEmulation.put("deviceName", "iPhone 14 Pro Max");
        options.setExperimentalOption("mobileEmulation", mobileEmulation);
        return options;
    }

    /**
     * Emulates custom responsive device metrics with touch action simulation.
     */
    public static ChromeOptions getCustomMobileMetrics() {
        ChromeOptions options = new ChromeOptions();
        Map<String, Object> deviceMetrics = new HashMap<>();
        deviceMetrics.put("width", 390);
        deviceMetrics.put("height", 844);
        deviceMetrics.put("pixelRatio", 3.0);
        deviceMetrics.put("touch", true);

        Map<String, Object> mobileEmulation = new HashMap<>();
        mobileEmulation.put("deviceMetrics", deviceMetrics);
        mobileEmulation.put("userAgent", "Mozilla/5.0 (iPhone; CPU iPhone OS 17_0 like Mac OS X)");

        options.setExperimentalOption("mobileEmulation", mobileEmulation);
        return options;
    }
}
```

16.12 Screen Recording & Automated Failure Screenshots

1. Enterprise Test Evidence Pipelines

In large-scale CI suites (5,000+ tests), capturing screenshots or videos for **every test** causes:

- Disk exhaustion on CI agents (200MB+ per suite).
- S3 upload latency that bottlenecks build pipelines.

Senior SDET Rule: Only capture screenshots and save video recordings **ON FAILURE**. Screenshots must be captured directly inside a TestNG `ITestListener` or JUnit 5 `TestWatcher` hook and attached to Allure / Extent

reports.

2. Production-Ready Failure Evidence Listener

```
package com.enterprise.automation.core.listeners;

import org.openqa.selenium.OutputType;
import org.openqa.selenium.TakesScreenshot;
import org.openqa.selenium.WebDriver;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.testng.ITestListener;
import org.testng.ITestResult;

import java.io.File;
import java.io.FileOutputStream;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public final class TestFailureListener implements ITestListener {

    private static final Logger log = LoggerFactory.getLogger(TestFailureListener.class);
    private static final String ARTIFACTS_DIR = "target/failure-artifacts";

    @Override
    public void onTestFailure(ITestResult result) {
        log.error("Test [{}]#{} FAILED. Capturing diagnostic artifacts...",
            result.getTestClass().getName(), result.getName());

        Object testInstance = result.getInstance();
        // Assuming test classes expose a getDriver() method or DriverFactory
        WebDriver driver = extractDriver(testInstance);

        if (driver != null) {
            captureScreenshot(driver, result.getName());
            capturePageSource(driver, result.getName());
        } else {
            log.warn("Could not retrieve active WebDriver instance for screenshot capture.");
        }
    }

    private void captureScreenshot(WebDriver driver, String testName) {
        try {
            byte[] screenshotBytes = ((TakesScreenshot) driver).getScreenshotAs(OutputType.BYTES);
            Path directory = Paths.get(ARTIFACTS_DIR, "screenshots");
            Files.createDirectories(directory);

            String timestamp = LocalDateTime.now().format(DateTimeFormatter.ofPattern("yyyyMMdd_"));
            Path destination = directory.resolve(testName + "_" + timestamp + ".png");

            try (FileOutputStream fos = new FileOutputStream(destination.toFile())) {

```

```

        fos.write(screenshotBytes);
    }
    log.info("Saved failure screenshot: {}", destination.toAbsolutePath());
} catch (IOException | RuntimeException e) {
    log.error("Failed to capture screenshot: {}", e.getMessage(), e);
}
}

private void capturePageSource(WebDriver driver, String testName) {
    try {
        String pageSource = driver.getPageSource();
        Path directory = Paths.get(ARTIFACTS_DIR, "pagesource");
        Files.createDirectories(directory);

        Path destination = directory.resolve(testName + ".html");
        Files.writeString(destination, pageSource);
        log.info("Saved failure DOM page source: {}", destination.toAbsolutePath());
    } catch (Exception e) {
        log.error("Failed to dump DOM page source: {}", e.getMessage());
    }
}

private WebDriver extractDriver(Object testInstance) {
    try {
        // Reflective lookup to decouple listener from test inheritance
        var field = testInstance.getClass().getSuperclass().getDeclaredField("driver");
        field.setAccessible(true);
        return (WebDriver) field.get(testInstance);
    } catch (Exception e) {
        return null;
    }
}
}

```

16.13 High-Stakes Senior Advanced Scenarios Interview Questions & Spoken Solutions

Q1: "Why does `driver.getWindowHandles()` return a Set instead of a List, and how do you guarantee you switch to the correct child window when 5 popups open simultaneously?"

Spoken Solution:

"Under the W3C WebDriver specification, window handles are distinct unique identifier strings representing browsing contexts. The Java bindings return `Set<String>` to denote uniqueness without positional order guarantees. Never rely on iteration order.

To handle multiple popups deterministically, I take a snapshot of existing handles before triggering the action, poll until the set size increments, and find the difference. If multiple windows open concurrently, I switch to each candidate handle and validate explicit contextual metadata—such as page title substring, URL regex patterns,

or the presence of a specific domain-level element—before executing the task. Once finished, I close the child context and restore focus back to the parent handle stored at the start."

Q2: "You have a modern React app with 3 nested iframes. The test intermittently fails with `NoSuchElementException` inside the innermost iframe. What is your diagnostic triage playbook?"

Spoken Solution:

"First, I check if the failure is caused by an asynchronous loading race condition. Standard `findElement` does not wait for an iframe's internal DOM document to load. I replace direct switching with `ExpectedConditions.frameToBeAvailableAndSwitchToIt()` applied sequentially for each parent iframe in the hierarchy.

Second, I verify that context leaks aren't occurring: before traversing down, I call `driver.switchTo().defaultContent()` to guarantee the driver is evaluated from the root document.

Third, if the iframes are cross-origin, browser security sandboxes may delay the child document initialization. I inspect network waterfalls to verify whether the inner iframe's source URL completed loading. If dynamically injected, I use a recursive traversal engine that scans frames dynamically while handling `StaleElementReferenceException`."

Q3: "What is the difference between legacy `--headless` and Chrome 109's `--headless=new`, and what headless-specific bugs have you resolved in enterprise CI pipelines?"

Spoken Solution:

"Legacy headless was a stripped-down browser variant that used a different codebase from headed Chrome. It had significant rendering divergences: CSS media queries evaluated differently, extensions were unsupported, and the default viewport was restricted to 800x600.

`--headless=new` leverages the exact same production rendering architecture as headed Chrome. The primary production bugs I've solved are:

1. Viewport collapses where responsive designs hid desktop navigation into mobile drawers—fixed by passing `--window-size=1920,1080`.
2. Missing system fonts in Linux Docker containers resulting in empty element text and collapsed bounding boxes—resolved by installing Microsoft and Liberation font packages in our Docker base images.
3. Shared memory crashes (`/dev/shm` exhaustion in Docker) solved with `--disable-dev-shm-usage`."

Q4: "How do you bypass UI login across a 5,000-test suite to optimize execution speed while still ensuring user authorization validity?"

Spoken Solution:

"We implement API-seeded session injection. Instead of performing UI-based login for every test, we send a direct HTTP POST to our authentication endpoint using `RestAssured` to obtain the session token or JWT.

In `WebDriver`, we navigate once to a fast static asset on the target origin—like `/robots.txt` or `/favicon.ico`—to establish domain context. We inject the authentication cookie via `driver.manage().addCookie()` and set any JWT tokens in `localStorage` via `JavascriptExecutor`. Finally, we navigate directly to the target application route. This drops login time from 5 seconds to 150 milliseconds per test, saving hundreds of CI computing hours each week."

Q5: "When should you use `JavascriptExecutor.executeScript()` over native `WebDriver` interactions, and what are the trade-offs?"

Spoken Solution:

"Native `WebDriver` interactions are preferred because they simulate true end-user behavior: checking element visibility, hit-testing against obscuring elements, and triggering native input events. Bypassing them risks missing customer-facing bugs like an overlay blocking a button.

However, `JavascriptExecutor` is essential in specific architectural edge cases: handling hidden `<input type='file'>` elements for file uploads, interacting with Shadow DOM roots across older browsers, extracting pseudo-element CSS values (like `::before` icons), and performing centered scrolling (`scrollIntoView({block: 'center'})`) to prevent sticky headers from intercepting clicks."

Q6: "How does Selenium 4's CDP support improve testing over Selenium 3 when dealing with network simulation and security permissions?"

Spoken Solution:

"In Selenium 3, simulating network conditions like 3G throttling or offline mode required external proxies like `BrowserMob Proxy`, which introduced networking bottlenecks and certificate installation overhead. Browser permissions like camera or geolocation had to be hardcoded statically in browser capabilities.

Selenium 4 connects directly to Chromium via `WebSocket` using the `Chrome DevTools Protocol`. We can dynamically call `Network.emulateNetworkConditions` to simulate flaky connections, offline PWA caching, and response mocks on the fly. Furthermore, we can grant or revoke browser permissions dynamically during test execution via `Browser.grantPermissions` without restarting the browser instance."

Q7: "How do you handle native OS authentication popups (HTTP Basic 401) in modern Selenium?"

Spoken Solution:

"HTTP 401 challenges are native OS authentication prompts, not JavaScript alerts, so `driver.switchTo().alert()` will throw `NoAlertPresentException`. Embedding credentials in the URL (`https://user:pass@domain.com`) is deprecated and blocked in modern Chromium for subresources.

In Selenium 4, the standard approach is using the `HasAuthentication` interface. We register credentials using `((HasAuthentication) driver).register(UsernameAndPassword.of("username", "password"))`. Under the hood, this uses CDP or WebDriver BiDi network listeners to automatically inject HTTP Authorization: Basic headers whenever a 401 challenge is intercepted."

Q8: "In a multi-threaded parallel suite, how do you prevent race conditions when manipulating cookies and window handles across tests?"

Spoken Solution:

"Race conditions occur when tests share browser instances or JVM driver references. We prevent this through strict thread isolation:

1. We store all `WebDriver` instances in a `ThreadLocal<WebDriver>` wrapper managed by a clean lifecycle factory.
2. Each parallel thread runs a completely independent browser process with an isolated user profile and temporary cache directory.
3. Cookie manipulation and window switching operate strictly within that thread's browsing context. Because memory spaces, network sockets, and browser processes are completely isolated, tests can add cookies or close windows without any cross-thread interference."